

BAO CAO KIEN THUC VA LOI GIAI BAI TAP LON: SMART BIKE

Tai lieu nay tong hop chi tiet toan bo kien thuc chuyen mon, phan tich ky thuat va loi giai hoan thien cho 5 Tasks trong bai tap lon he thong dieu khien xe dap dien thong minh (Smart Bike).

TASK 1: STRUCT, UNION & BIT-FIELDS (DINH NGHIA THANH GHI 16-BIT)

1. Yeu cau va Phan tich Ky thuat

Trong cac he thong nhung, viec toi uu hoa bo nho luu tru va giao tiep giua cac thiet bi ngoai vi thuong yeu cau doc/ghi truc tiep cac thanh ghi phan cung o cap do bit. Su dung phuong phap dich bit (bit-masking) truyen thong de cau hinh de dan den sai sot va kho bao tri. Do do, viec su dung phuong phap **Bit-fields** ket hop voi **Union** giup anh xa truc tiep cac truong du lieu vao mot thanh ghi duy nhat.

Yeu cau cua thanh ghi 16-bit raw_value duoc phan bo tu bit thap den bit cao (Little-endian) nhu sau:

- **PWR_ON**: 1 bit -> Chiem vi tri bit [0]
- **ASSIST_LEVEL**: 2 bit -> Chiem vi tri bit [2:1]
- **LIGHT_BRIGHT**: 4 bit -> Chiem vi tri bit [6:3]
- **RESERVED**: 9 bit -> Chiem cac bit con lai [15:7] de dam bao tong do rong la 16-bit.

2. Ma nguon Trien khai

```
typedef union {
    uint16_t raw_value;
    struct {
        uint16_t PWR_ON      : 1;
        uint16_t ASSIST_LEVEL : 2;
        uint16_t LIGHT_BRIGHT : 4;
        uint16_t RESERVED    : 9;
    } fields;
} Bike_Status_t;
```

TASK 2: FUNCTION POINTERS & TABLE-DRIVEN

ARCHITECTURE

1. Ly thuyet va Uu diem kien truc

Kien truc **Table-Driven** (Kien truc huong bang) ket hop voi con tron ham (Function Pointers) la mot thiet ke kinh dien de thay the cho cac khoi dieu khien rã nhanh phuc tap dung if-else hoac switch-case dai dong.

- **Thoi gian dap ung hang so $O(1)$:** He thong truy xuat truc tiep den ham can goi thong qua chi so mang (index), khong ton thoi gian so sanh tuan tu. Dieu nay dac biet quan trong trong cac thiet bi nhung yeu cau tinh thoi gian thuc (Real-time).
- **De dang mo rong (Scalability):** Khi he thong co them che do bieu dien hoac tinh nang moi, lap trinh vien chi can khai bao them ham va them con tron ham vao mang ma khong can thay doi logic dieu khien goc trong luong thuc thi chinh.

2. Ma nguon Trien khai

```
typedef void (*Drive_Mode_Handler_t) (void);
#define DRIVE_MODE_COUNT 3U

Drive_Mode_Handler_t drive_modes[DRIVE_MODE_COUNT] = {
    drive_eco,
    drive_normal,
    drive_sport
};

// Luong goi trong main dam bao an toan bo nho:
if ((my_bike.fields.ASSIST_LEVEL < DRIVE_MODE_COUNT) &&
    (drive_modes[my_bike.fields.ASSIST_LEVEL] != NULL)) {
    (*drive_modes[my_bike.fields.ASSIST_LEVEL]) ();
} else {
    printf("Invalid assist level.\n");
}
```

TASK 3: ARCHITECTURE CALLBACKS (CO CHE CON TRON HAM NGUOC)

1. Ban chat cua Callback

Co che **Callback** cho phep mot module tang duoi (nhu Driver hoac Hardware Abstract Layer -

HAL) có thể gọi ngược lên một hàm xử lý thuộc module tầng trên (Application Layer) khi xuất hiện một sự kiện đặc biệt hoặc ngắt phản ứng mà không cần biết trước tên hàm do vào lúc biên dịch.

Điều này tạo nên tính chất **Low Coupling** (Liên kết lỏng lẻo) giúp các lớp phần mềm độc lập, tăng khả năng tái sử dụng code trên nhiều dòng chip khác nhau.

2. Mã nguồn Triển khai

```
void Battery_Monitor(uint8_t battery_temp) {
    if ((battery_temp > 40) && (overheat_cb != NULL)) {
        (*overheat_cb)();
    }
}
```

TASK 4: MEMORY MAP & CRASH LAB

1. Thiết kế mã nguồn gây tràn ngăn xếp (Stack Overflow)

Để giả lập hiện tượng sụp đổ hệ thống do cạn kiệt vùng nhớ Stack, phương pháp tối ưu nhất là sử dụng để quy vô hạn kết hợp cấp phát mang cục bộ có kích thước lớn trên mỗi frame hàm:

```
void stack_overflow_recursive(uint32_t depth) {
    uint8_t stack_load[1024]; // Cấp phát 1KB trên Stack mỗi lần gọi
    // de quy
    stack_load[0] = (uint8_t)(depth & 0xFF);

    volatile uint32_t keep_running = 1U;
    if (keep_running != 0U) {
        stack_overflow_recursive(depth + 1U);
    }
}

void crash_simulation(void) {
    stack_overflow_recursive(0U);
}
```

2. [Tu luan 4.2] Phan bo vi tri cac bien vao vung nho

Ten Bien	Phan Vung Bo Nho	Giai Thich Chi Tiet Gia Tri & Vi Tri
BIKE_MODEL	.rodata (Read-Only Data)	La bien hang so cuc bo hoac toan cuc co tu khoa const va da khoi tao gia tri chuoi. Vung nay nam o ROM/Flash de tranh bi thay doi trong qua trinh chay.
total_odometer	.bss (Block Started by Symbol)	La bien toan cuc chua khoi tao (hoac khoi tao bang 0). Vung nay nam o RAM va duoc ma Startup xoa ve 0 luc khoi dong chuong trinh.
current_speed	Stack	La bien cuc bo thong thuong duoc khai bao ben trong ham main(). Dynamic allocation/deallocation theo thoi gian song cua ham.

3. [Tu luan 4.3] Hien tuong say ra khi say ra su co tran Stack

- **Tren May tinh (Moi truong OS nhu Linux/Windows):** He dieu hanh quan ly bo nho bang Virtual Memory va co co che bao ve (MMU). Khi Stack Pointer vuot khoi trang bao ve, OS se ngay lap tuc cham dut tien trinh va dua ra thong bao loi: Segmentation fault (core dumped) de bao ve tinh toan ven cua cac tien trinh khac.
- **Tren Vi dieu khien (Embedded Hardware thuc te - ARM Cortex-M):** Khong co OS dung nghia bao ve. Khi Stack bi tran, con tron ngan xep SP se ghi de vao vung du lieu cua cac bien toan cuc (.data, .bss) hoac vuot ra ngoai vung dia chi RAM. CPU se phat hien vi phamphan cung va kich hoat ngoai le nghiem trong, lap tuc nhay vao trinh phuc vu ngat: HardFault_Handler() de treo he thong hoac Reset chip.

TASK 5: TOI UU HOA DUNG LUONG VOI LINKER SCRIPT (TU LUAN)

1. Tinh huong dat ra

Chip co Flash con trong 20 KB, RAM con trong 60 KB. Yeu cau khai bao mot mang lich su chuyen di co kich thuoc: 20,000 phan tu kieu uint16_t (tuong duong: $20,000 \times 2 \text{ bytes} = 40,000 \text{ bytes}$ $\approx 40 \text{ KB}$).

2. Lua chọn và Giải thích bản chất Linker

Giai pháp: Bước phải chọn **Cách 1** (`uint16_t trip_history[20000] = {0};`) để chương trình có thể biên dịch và nạp thành công vào chip.

3. Phân tích chi tiết hành vi của Linker:

- **Đối với Cách 1 (= {0};):** Mảng được khởi tạo giá trị bằng 0, vì vậy Linker sẽ phân bổ mảng này vào phân vùng **.bss** của RAM.
Có thể lưu trữ: Pháo file `.bin/.hex` nạp vào Flash, phân vùng `.bss` **không chiếm dung lượng không gian thực tế**. Linker chỉ ghi nhận 2 thông số: Địa chỉ bắt đầu và Độ lớn (40 KB) của khối `.bss` vào Flash (chỉ tốn vài bytes). Khi chip khởi động, đoạn mã Startup sẽ lấy 2 thông số này và thực hiện vòng lặp xóa trang 40 KB RAM về 0.
Kết quả: Flash tốn thêm ~0 KB (< 20 KB trong), RAM tốn 40 KB (< 60 KB trong) -> **Biên dịch và nạp thành công**.
- **Đối với Cách 2 (= {1, 2, 3...};):** Mảng được khởi tạo với các giá trị khác không, bước Linker phải phân bổ nó vào phân vùng **.data**.
Có thể lưu trữ: Toàn bộ thông tin giá trị khởi tạo của 40 KB dữ liệu này **phải được lưu trữ đầy đủ trực tiếp trong bộ nhớ Flash**. Khi cấp nguồn, mã Startup sẽ sao chép nguyên vẹn 40 KB này từ Flash sang vùng RAM tương ứng.
Kết quả: Flash bắt buộc phải có dung lượng 40 KB để chứa mảng này, nhưng Flash hiện tại chỉ còn trong 20 KB -> Gặp lỗi nghiêm trọng lúc Link: Linker Error: Flash Overflow / Region FLASH overflowed -> **Biên dịch thất bại**.